

DATA WAREHOUSE IMPLEMENTATION

- Data warehouses contain huge volumes of data
- OLAP servers demand that decision support queries be answered in the order of seconds
- It is crucial for data warehouse systems to support
 - highly efficient cube computation techniques,
 - access methods, and
 - query processing techniques

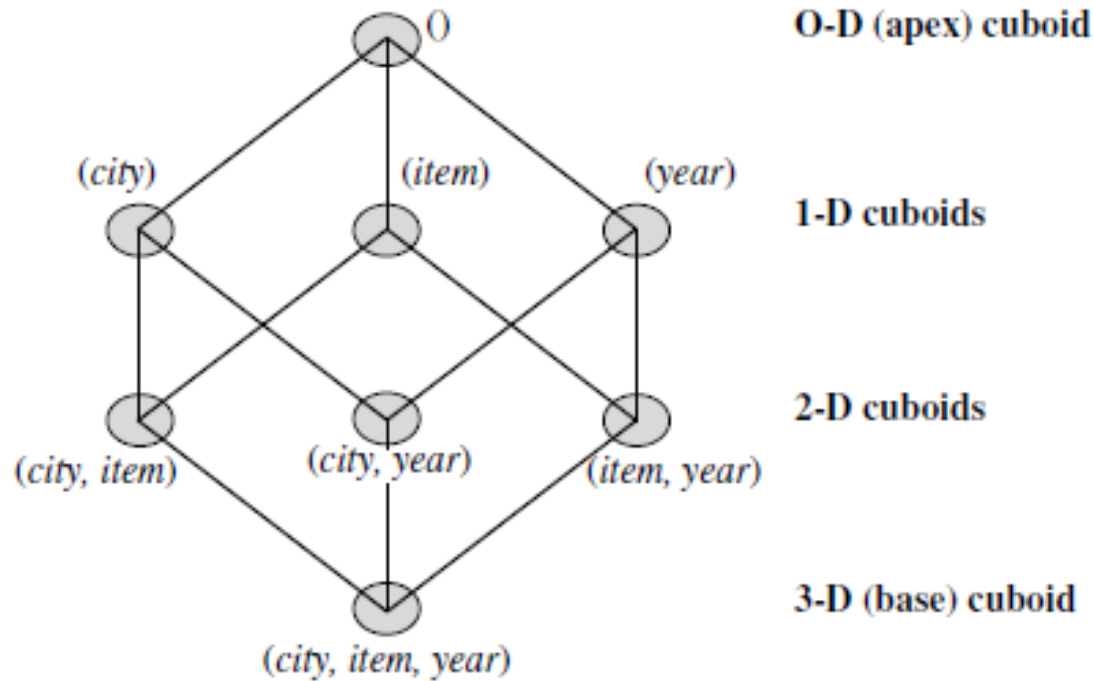
Efficient Data Cube Computation: An Overview

- At the core of multidimensional data analysis is the efficient computation of aggregations across many sets of dimensions
- In SQL terms, the aggregations are referred to as group-by's
- Each group-by can be represented by a *cuboid*, where the set of group-by's forms a lattice of cuboids defining a data cube

The compute cube Operator and the Curse of Dimensionality

- One approach to cube computation extends SQL so as to include a compute cube operator
- The compute cube operator computes aggregates over all subsets of the dimensions specified in the operation
- This can require excessive storage space, especially for large numbers of dimensions

- A data cube is a lattice of cuboids
- to create a data cube for *AllElectronics* sales that contains the *city*, *item*, *year*, and *sales in dollars*:
- Total number of cuboids, or groupby's, that can be computed for this data cube is $2^3 = 8$
- The possible group-by's are:
 $\{(city, item, year), (city, item), (city, year), (item, year), (city), (item), (year), ()\}$, where $()$ means that the group-by is empty (i.e., the dimensions are not grouped)



-
- ⌋ Lattice of cuboids, making up a 3-D data cube. Each cuboid represents a different group-by. The base cuboid contains *city*, *item*, and *year* dimensions.

- The **base cuboid** contains all three dimensions, *city*, *item*, and *year*.
- It can return the total sales for any combination of the three dimensions
- The base cuboid is the least generalized (most specific) of the cuboids
- The **apex cuboid**, or 0-D cuboid, refers to the case where the group-by is empty
- It contains the total sum of all sales
- The Apex cuboid is the most generalized (least specific) of the cuboids, and is often denoted as all
- If we start at the apex cuboid and explore downward in the lattice, this is equivalent to drilling down within the data cube
- If we start at the base cuboid and explore upward, this is akin to rolling up.

- An SQL query containing no group-by is a *zero dimensional operation*
 - e.g., “*compute the sum of total sales*”
- An SQL query containing one group-by is a *one-dimensional operation*
 - e.g., “*compute the sum of sales, group-by city*”
- A cube operator on n dimensions is equivalent to a collection of group-by statements, one for each subset of the n dimensions
- cube operator is the n -dimensional generalization of the group-by operator

- Data cube definition:
- `define cube sales cube [city, item, year]: sum(sales in dollars)`
- For a cube with n dimensions, there are a total of 2^n cuboids, including the base cuboid

`compute cube sales_cube`

- Statement explicitly instruct the system to compute the sales aggregate cuboids for all eight subsets of the set $\{city, item, year\}$, including the empty subset

- Online analytical processing may need to access different cuboids for different queries
- It is good idea to compute in advance all or at least some of the cuboids in a data cube
- Precomputation leads to fast response time and avoids some redundant computation
- **Challenge:** required storage space may explode if all the cuboids in a data cube are precomputed, especially when the cube has many dimensions
- Storage requirements are even more excessive when many of the dimensions have associated concept hierarchies, each with multiple levels
- This problem is referred to as the **curse of dimensionality**

No. of cuboids in an n-dimensional data cube:

- If there are no hierarchies associated with each dimension then it is 2^n
- If hierarchies are associated with each dimension, (e.g: *time: day < month < quarter < year*)

$$\text{Total number of cuboids} = \prod_{i=1}^n (L_i + 1)$$

- where L_i is the number of levels associated with dimension i .
- 1 is added to L_i to include the *virtual* top level, all

- Example:
- time dimension has five levels (including all)
- If the cube has 10 dimensions and each dimension has five levels (including all),
- The total number of cuboids that can be generated is 5^{10}

- It is unrealistic to precompute and materialize all of the cuboids that can possibly be generated for a data
- If there are many cuboids, and these cuboids are large in size, a more reasonable option is *partial materialization*
 - to materialize only *some* of the possible cuboids that can be generated

Partial Materialization: Selected Computation of Cuboids

There are three choices for data cube materialization given a base cuboid:

No materialization:

- Do not precompute any of the cuboids
- This leads to computing expensive multidimensional aggregates on-the-fly, which can be extremely slow

Full materialization:

- Precompute all of the cuboids
- The resulting lattice of computed cuboids is referred to as the *full cube*.
- This choice typically requires huge amounts of memory space in order to store all of the precomputed cuboids

Partial materialization:

- Selectively compute a proper subset of the whole set of possible Cuboids
- Alternatively, we may compute a subset of the cube, which contains only those cells that satisfy some user-specified criterion, such as where the tuple count of each cell is above some threshold
- Represents an interesting trade-off between storage space and response time

Partial materialization of cuboids should consider three factors:

- Identify the subset of cuboids or subcubes to materialize
- Exploit the materialized cuboids or subcubes during query processing
- Efficiently update the materialized cuboids or subcubes during load and refresh.

Indexing OLAP Data

- To facilitate efficient data accessing, most data warehouse systems support index structures
- OLAP data can be indexed using
 - **Bitmap Index**
 - **Join Index**

Bitmap Index

- Popular method because it allows quick searching in data cubes
- It is an alternative representation of the *record ID (RID)* list
- In the bitmap index for a given attribute, there is a distinct bit vector, B_v , for each value v in the attribute's domain
- If a given attribute's domain consists of n values, then n bits are needed for each entry in the bitmap index
- If the attribute has the value v for a given row in the data table, then the bit representing that value is set to 1 in the corresponding row of the bitmap index. All other bits for that row are set to 0

- **Example:**
- Consider AllElectronics data warehouse
- Let dimension *item* at the top level has four values- “home entertainment,” “computer,” “phone,” and “security.”
- Each value (e.g., “computer”) is represented by a bit vector in the *item* bitmap index table
- If the cube is stored as a relation table with 100,000 rows, because the domain of *item* consists of four values, the bitmap index table requires four bit vectors (or lists), each with 100,000 bits
- The diagram shows a base (data) table containing the dimensions *item* and *city*, and its mapping to bitmap index tables for each of the dimensions

Base table

<i>RID</i>	<i>item</i>	<i>city</i>
R1	H	V
R2	C	V
R3	P	V
R4	S	V
R5	H	T
R6	C	T
R7	P	T
R8	S	T

item bitmap index table

<i>RID</i>	H	C	P	S
R1	1	0	0	0
R2	0	1	0	0
R3	0	0	1	0
R4	0	0	0	1
R5	1	0	0	0
R6	0	1	0	0
R7	0	0	1	0
R8	0	0	0	1

city bitmap index table

<i>RID</i>	V	T
R1	1	0
R2	1	0
R3	1	0
R4	1	0
R5	0	1
R6	0	1
R7	0	1
R8	0	1

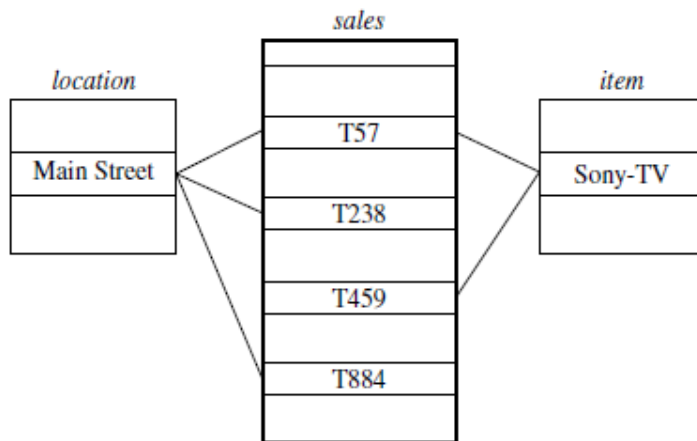
Note: H for “home entertainment,” C for “computer,” P for “phone,” S for “security,” V for “Vancouver,” T for “Toronto.”

- It is especially useful for low-cardinality domains
- Comparison, join, and aggregation operations are reduced to bit arithmetic, which substantially reduces the processing time
- leads to significant reductions in space and input/output (I/O) since a string of characters can be represented by a single bit
- For higher-cardinality domains, the method can be adapted using compression techniques

Join index

- Gained popularity from its use in relational database query processing
- Join indexing registers the joinable rows of two relations from a relational database
- If two relations $R(RID, A)$ and $S(B, SID)$ join on the attributes A and B , then the join index record contains the pair (RID, SID) , where RID and SID are record identifiers from the R and S relations, respectively
- The join index records can identify joinable tuples without performing costly join operations
- It is especially useful for maintaining the relationship between a foreign key and its matching primary keys, from the joinable relation
- In data warehouses, join index relates the values of the dimensions of a star schema to rows in the fact table
- Join indices may span multiple dimensions to form **composite join indices**

- Consider a star schema for *AllElectronics* of the form
- “*sales_star* [*time, item, branch, location*]: *dollars sold* D *sum (sales in dollars).*”
- Example for join index relationship between the *sales* fact table and the *location* and *item* dimension tables
- Example: the “Main Street” value in the *location* dimension table joins with tuples T57, T238, and T884 of the *sales* fact table
- Similarly, the “Sony-TV” value in the *item* dimension table joins with tuples T57 and T459 of the *sales* fact table
- To further speed up query processing, the join indexing and the bitmap indexing methods can be integrated to form **bitmapped join indices**



Linkages between a *sales* fact table and *location* and *item* dimension tables.

Join index table for
location/sales

<i>location</i>	<i>sales_key</i>
...	...
Main Street	T57
Main Street	T238
Main Street	T884
...	...

Join index table for
item/sales

<i>item</i>	<i>sales_key</i>
...	...
Sony-TV	T57
Sony-TV	T459
...	...

Join index table linking
location and *item* to *sales*

<i>location</i>	<i>item</i>	<i>sales_key</i>
...	...	...
Main Street	Sony-TV	T57
...	...	...

Join index tables based on the linkages between the *sales* fact table and the *location* and *item*

Efficient Processing of OLAP Queries

- The purpose of materializing cuboids and constructing OLAP index structures is to speed up query processing in data cubes
- Given materialized views, query processing should proceed as follows:
- **Determine which operations should be performed on the available cuboids**
 - Involves transforming any selection, projection, roll-up (group-by), and drill-down operations specified in the query into corresponding SQL and/or OLAP operations
- **Determine to which materialized cuboid(s) the relevant operations should be applied**
 - Involves identifying all of the materialized cuboids that may potentially be used to answer the query, pruning the set using knowledge of “dominance” relationships among the cuboids, estimating the costs of using the remaining materialized cuboids, and selecting the cuboid with the least cost

- Consider a data cube for *AllElectronics* of the form
“sales cube [*time, item, location*]: sum(*sales in dollars*).”
- dimension hierarchies:
 - *day* < *month* < *quarter* < *year* for *time*
 - *item name* < *brand* < *type* for *item*;
 - *street* < *city* < *province or state* < *country* for *location*
- Query to be processed is on {*brand, province_or_state*}, with the selection constant “*year = 2010*.”
- cuboid 1: {*year, item name, city*}
- cuboid 2: {*year, brand, country*}
- cuboid 3: {*year, brand, province or state*}
- cuboid 4: {*item name, province or state*}, where *year = 2010*
- “Which of these four cuboids should be selected to process the query?”

- Cuboid 2 cannot be used because *country* is a more general concept than *province or state*
- Cuboids 1, 3, and 4 can be used to process the query because
- They have the same set or a superset of the dimensions in the query,
- The selection clause in the query can imply the selection in the cuboid, and
- The abstraction levels for the *item* and *location* dimensions in these cuboids are at a finer level than *brand* and *province or state*, respectively

- Cost Factors:
- It is likely that using cuboid 1 would cost the most because both *item name* and *city* are at a lower level than the *brand* and *province or state* concepts specified in the query.
- If there are not many *year* values associated with *items* in the cube, but there are several *item names* for each *brand*, then cuboid 3 will be smaller than cuboid 4, and thus cuboid 3 should be chosen to process the query.
- However, if efficient indices are available for cuboid 4, then cuboid 4 may be a better choice. Therefore, some cost-based estimation is required to decide which set of cuboids should be selected for query processing

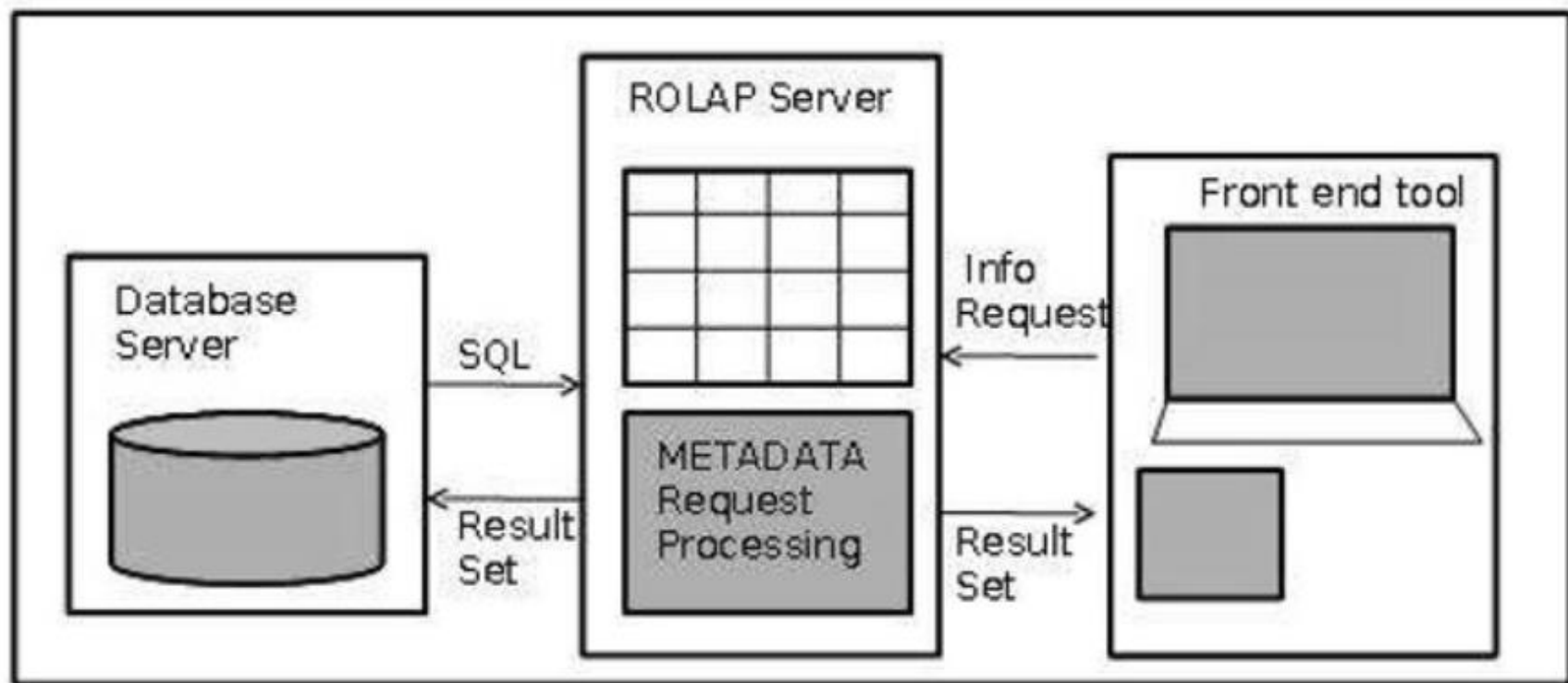
OLAP Server Architectures

- OLAP servers present business users with multidimensional data from data warehouses or data marts, without concerns regarding how or where the data are stored
- However, the physical architecture and implementation of OLAP servers must consider data storage issues
- Implementations of a warehouse server for OLAP processing include the following architectures:
 - Relational OLAP (ROLAP) servers
 - Multidimensional OLAP (MOLAP) servers
 - Hybrid OLAP (HOLAP) servers

Relational OLAP (ROLAP) servers

- Relational OLAP servers are placed between relational back-end server and client front-end tools
- To store and manage the warehouse data, the relational OLAP uses relational or extended-relational DBMS
- ROLAP includes the following :
 - Implementation of aggregation navigation logic
 - Optimization for each DBMS back-end
 - Additional tools and services
- A ROLAP database can be accessed through complex SQL queries to calculate information

- **Advantages:**
- ROLAP servers can be easily used with existing RDBMS
- ROLAP can handle large data volumes, but the larger the data, the slower the processing times
- Because queries are made on-demand, ROLAP does not require the storage and pre-computation of information
- **Disadvantages:**
- Potential performance constraints and scalability limitations that result from large and inefficient join operations between large tables



A ROLAP data store

- ROLAP stores data in columns and rows and retrieves the information on demand through user submitted queries
- Consider a summary fact table that contains both base fact data and aggregated data
- The schema is “<record identifier (RID), item, ..., day, month, quarter, year, dollars sold>”

Single Table for Base and Summary Facts

<i>RID</i>	<i>item</i>	<i>...</i>	<i>day</i>	<i>month</i>	<i>quarter</i>	<i>year</i>	<i>dollars_sold</i>
1001	TV	...	15	10	Q4	2010	250.60
1002	TV	...	23	10	Q4	2010	175.00
...	...	...	...	...	...	...	...
5001	TV	...	all	10	Q4	2010	45,786.08
...	...	...	...	...	...	...	...

- Tuples with an *RID* of 1001 and 1002 data are at the base fact level, where the sales dates are October 15, 2010, and October 23, 2010, respectively
- Tuple with an *RID* of 5001 is at a more general level of abstraction
- *day* value has been generalized to all, so that the corresponding *time* value is October 2010 i.e. the *dollars sold* amount shown is an aggregation representing the entire month of October 2010, rather than just October 15 or 23, 2010.
- The special value all is used to represent subtotals in summarized data

Multidimensional OLAP (MOLAP) servers

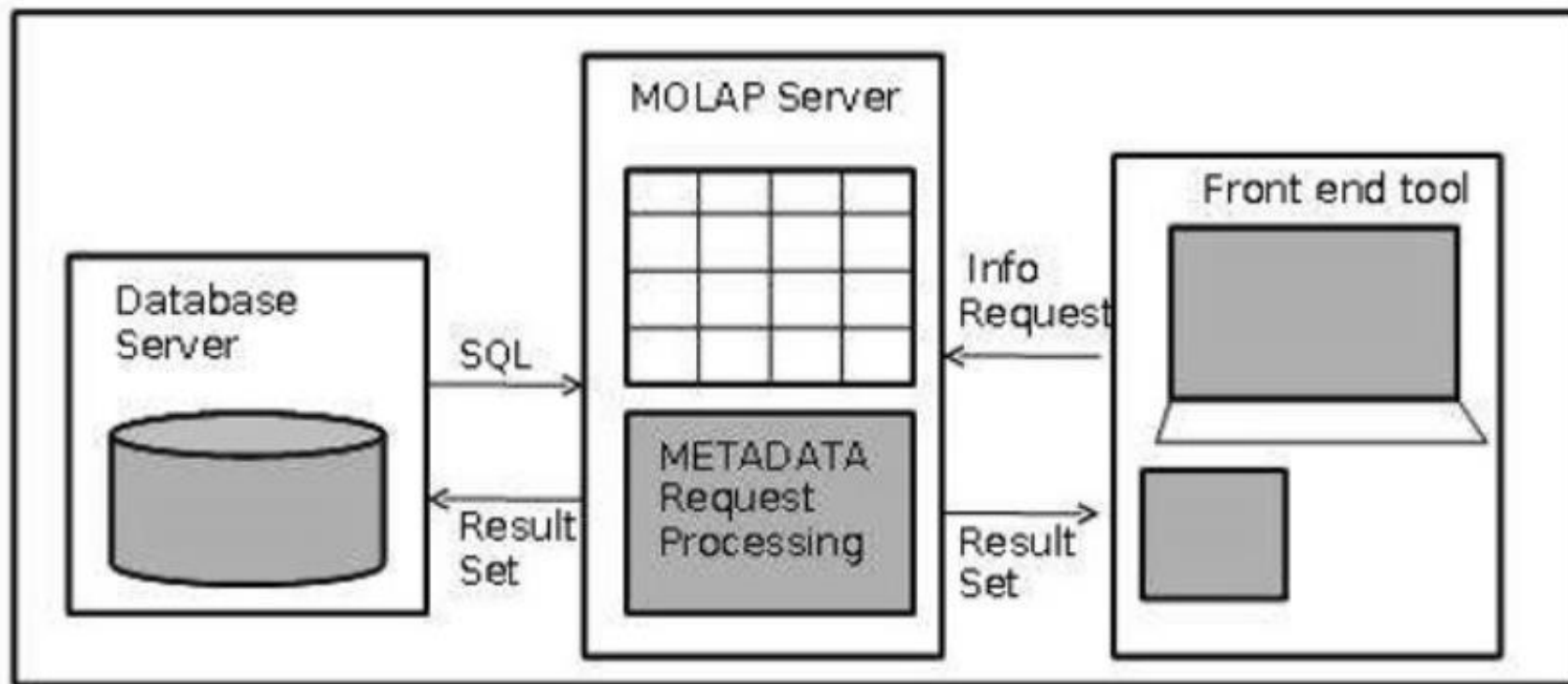
- MOLAP uses array-based multidimensional storage engines for multidimensional views of data
- They map multidimensional views directly to data cube array structures
- With multidimensional data stores, the storage utilization may be low if the dataset is sparse
- Therefore, many MOLAP servers use two levels of data storage representation to handle dense and sparse datasets
- Denser subcubes are identified and stored as array structures, whereas sparse subcubes employ compression technology for efficient storage utilization

Advantages:

- Data is pre-computed, pre-summarized, and stored
- Its simple interface makes MOLAP easy to use, even for inexperienced users
- Its speedy data retrieval makes it the best for “slicing and dicing” operations

Disadvantages:

- Less scalable than ROLAP, as it can handle a limited amount of data
- Storage utilization may be low if the data set is sparse



Hybrid OLAP (HOLAP) servers

- Combines ROLAP and MOLAP technology
- involves storing part of data in a ROLAP store and another part in a MOLAP store, thus developers get the benefits of both
- Gets benefited from the greater scalability of ROLAP and the faster computation of MOLAP
- HOLAP server may allow large volumes of detailed data to be stored in a relational database, while aggregations are kept in a separate MOLAP store
- This setup allows much more flexibility for handling data

